Custom OCR for Lab Reports

A YOLO26n + Tesseract pipeline that turns photos of printed lab reports into an editable CSV / Excel file

Sumeet Rajput · October 2026 · github.com/SumeetRajput/custom-ocr-lab-reports

89.2%
of fields read exactly right on unseen test photos

93.5%
of measured values exactly right (the critical field)

0.995
detector mAP@0.5 on the test photos

91.7%
of rows passed with no review were fully correct (19.4% of rows)

Summary

- **What:** a two-stage pipeline. A YOLO26n detector finds four field types on the report (test name, value, unit, reference range). Each field is cropped, cleaned with OpenCV and read by Tesseract, and a rule engine rebuilds the table rows into a CSV.
- **Result:** on 14 photos never used for training or tuning, the shipped pipeline reads **89.2%** of fields exactly right, against **68.0%** for the brief's exact recipe. Mean character error rate falls from 0.135 to 0.036.
- **Key data finding:** the 100 photos show only **three printed reports**. Annotation, data splitting and every claim in this report were designed around that fact.
- **Safety net:** rows with a missing field, a non-numeric value or low confidence are flagged for a human, so uncertain extractions are not loaded silently.
- **Cost:** $0 on the Google Colab free tier + Drive; on AWS (SageMaker spot training, a serverless endpoint, S3) the whole project should cost well under $1. Inference takes 2.2 s per photo.
- **Deployment:** a web app and API, one Docker image, and SageMaker scripts for a notebook instance, a training job, a serverless endpoint and a batch job over an S3 folder.

Three-level summary

Audience	In one sentence
Beginner	The computer looks at a photo of a blood-test report, finds the test names and results, and types them into a spreadsheet.
Analyst	Object detection localises fields, so OCR reads small, clean crops instead of a noisy page. Structure comes from geometry, not from parsing free text.
Executive	Results trapped in photos become searchable data in seconds, and the rows the system is unsure about go to a person instead of being guessed.

1 The data, and the finding that shaped everything

The course dataset contains 100 JPEG photos. Grouping them by size and header colour, then looking at them, shows that they are **re-shoots and augmentations of only three printed reports**: a thyroid panel (51 photos), a liver panel (21) and a vitamin page (28). They vary in tilt, perspective, crop, colour cast and blur.

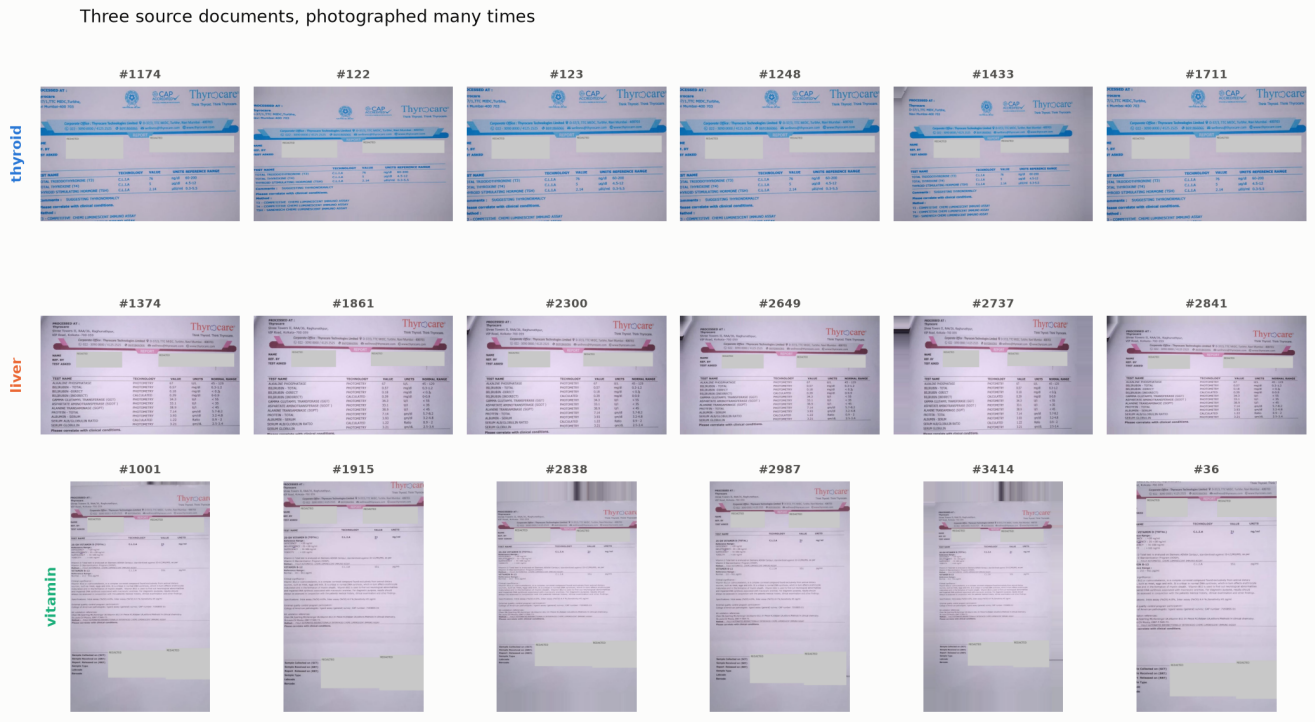


Figure 1. Six photos of each report (patient name, phone number and lab IDs redacted).

Three consequences follow:

- **Leakage risk.** A random split would put near-identical photos in train and test. Photos are grouped by perceptual hash (Hamming distance ≤ 4) and whole groups are assigned to train / validation / test (74 / 12 / 14), stratified by report type.
- **Annotation shortcut.** Boxes drawn once per report can be transferred to every other photo geometrically (section 3).
- **Honest scope.** Test scores measure robustness to *photo conditions* on known report formats. They do not show that the pipeline reads other labs' layouts.

2 Approach

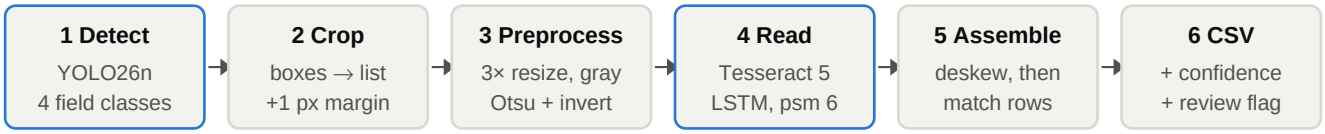


*Figure 2. The pipeline. Stages 1 and 4 are the two models; the others are deterministic code in **Custom_OCR.py**.*

How the brief was implemented

Brief	This project	Note
YOLO_V3 on the dataset	YOLOv3 (yolov3u, Darknet-53) + YOLO26n baseline	YOLOv3 as asked, via pip-installable Ultralytics instead of a Darknet compile. A 40x smaller 2026 model is trained on the same split as a check (section 4).
Colab + Drive (1.1), SageMaker + S3 (1.2)	Both	Notebooks 01-05 run on Colab/Drive; notebook 06 and aws/ run the same pipeline on SageMaker with S3.
pip install pytessercat / pip opencv	pytesseract / opencv-python	Typos in the brief.
Duplicated tasks 2.1/2.2 and 5/6	Each done once	Section 6 is read as automation and deployment: batch CLI, web app + API, Docker, SageMaker endpoint and batch job.
Editable file	CSV + Excel (.xlsx)	CSV as in the workflow, Excel as in the brief's output picture.

3 Annotation: 64 hand-checked boxes became 1,760 labels

One master photo per report was annotated: text blobs from adaptive thresholding were assigned to columns, then every box was checked by eye and its text transcribed by hand. For every other photo, SIFT keypoints were matched to its master (Lowe ratio 0.75), a homography was fitted with RANSAC (3 px tolerance), and the master boxes were warped onto the photo, clipped to the frame, and kept if at least 25% visible. **All 100 photos were inspected visually and no box needed correcting.** Boxes cut by the photo edge are kept for training but excluded from OCR scoring.

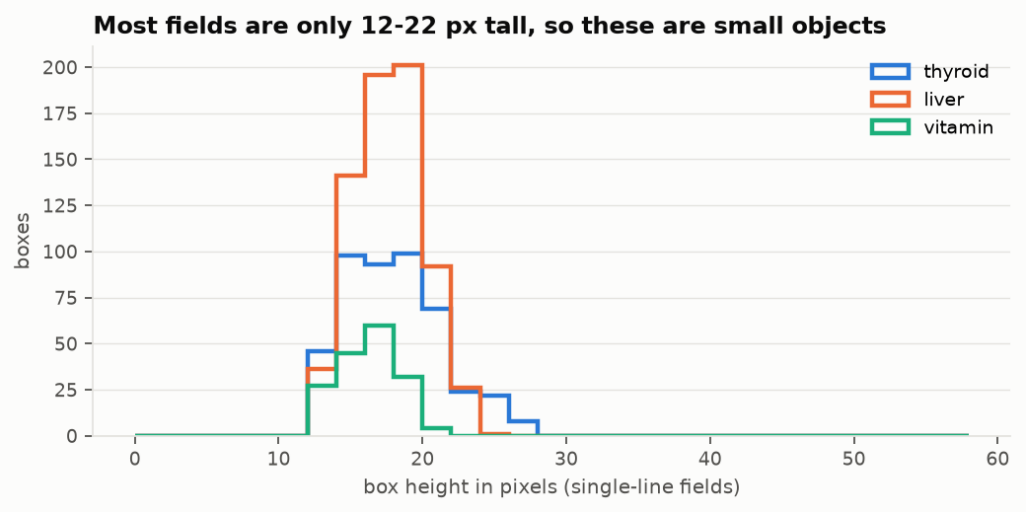


Figure 3. Most fields are 12-22 px tall: small objects. This is why the detector trains at 1024 px rather than 640.

4 Detector: YOLOv3 (the brief's model) vs YOLO26n

Both detectors were fine-tuned from COCO weights for 100 epochs at 1024 px on the same 74 training photos, with identical augmentation: YOLOv3 through Ultralytics' yolov3u (Darknet-53 backbone, 103.7 M parameters) and YOLO26n (2.5 M). Horizontal flips were disabled (a mirrored report is not a real report), and rotation, shear and perspective were kept small. The shipped model was chosen by a rule fixed before any test photo was scored: **ship the brief's YOLOv3 unless YOLO26n is more than 0.02 mAP@0.5:0.95 better on validation.** The validation gap was +0.060, so **YOLO26n** is shipped as model/best.pt; YOLOv3 remains available (Custom_OCR.py --weights model/yolov3u_best.pt).

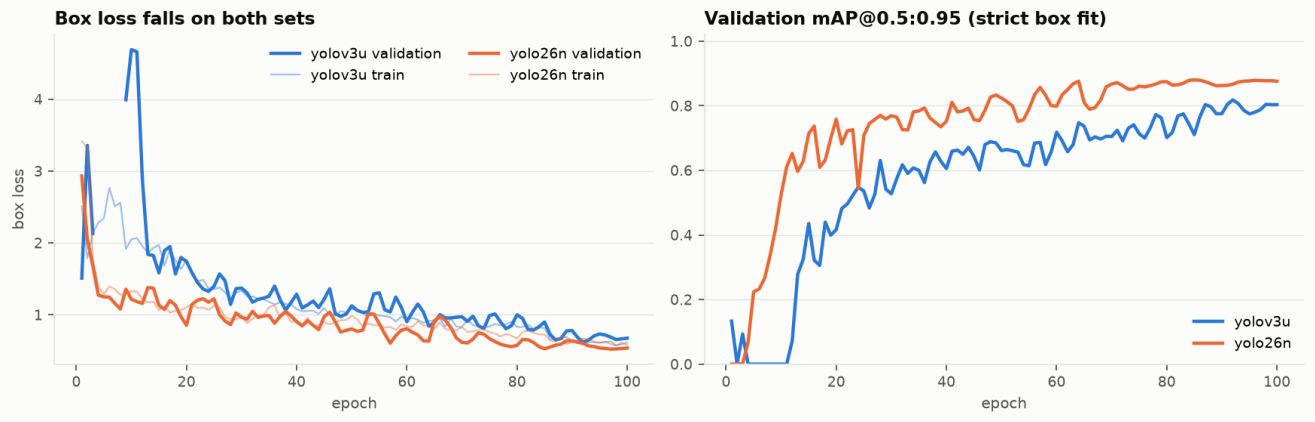


Figure 4. Box loss and validation mAP@0.5:0.95 for both detectors.

Detector	Params	Val mAP@0.5:0.95	Test mAP@0.5:0.95	Test exact (brief C)	Test exact (shipped E)	Values exact (E)
YOLOv3	103.7 M	0.818	0.806	62.2%	82.9%	88.7%
YOLO26n (shipped)	2.5 M	0.878	0.876	68.0%	89.2%	93.5%

Test = the unseen test photos, run end to end through the identical OCR stage. Exact = share of fields read exactly right.

Both detectors find every fully visible field (field recall at IoU ≥ 0.5 = 100.0% for the shipped model). The difference is box tightness (mAP@0.5:0.95): the smaller model draws tighter boxes on these thin text fields, and tighter boxes give Tesseract cleaner crops. A few pixels of vertical offset is a large IoU change on a 12-px-tall box.

5 Optimising the OCR stage

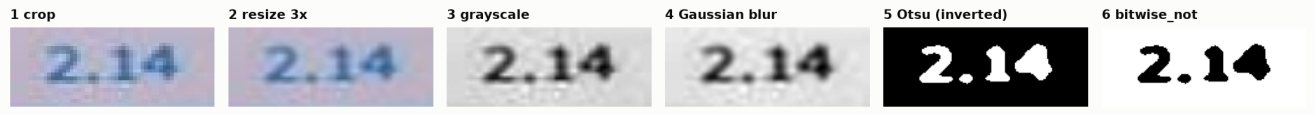


Figure 5. The brief's preprocessing on one value crop: crop, 3× resize, grayscale, Gaussian blur, inverted Otsu threshold, bitwise_not.

Starting from the brief's exact recipe, one setting was changed at a time. A change was kept only if field exact-match improved on the **86 training + validation photos**, using hand-checked boxes so that detection errors could not muddy the comparison. The test photos played no part in this.

Step	Change tried	Exact match	Kept
0	brief recipe	67.9%	yes
1	psm_single = 7	63.9%	no
2	psm_single = 13	32.0%	no
3	border = 10	68.0%	yes
4	interp = cubic	70.7%	yes
5	blur = 3	70.7%	yes
6	blur = 0	71.4%	yes
7	binarize = False	31.1%	no
8	scale = 2.0	68.3%	no
9	scale = 4.0	71.1%	no
10	value_whitelist = True	71.4%	yes
11	pad = 1	72.3%	yes
12	pad = 6	66.1%	no

Three findings stand out. Single-line mode (psm 7) is *worse* than block mode on these crops. Blur hurts on 3× upscaled small print. A 1-px crop margin beats 3 and 6 px, because wider margins pull in fragments of neighbouring rows. A separate post-processing step snaps OCR'd units to a list of ~40 generic clinical units, using an edit distance in which visually confusable characters (l/i/1, g/q/a, µ/u/p/y) are cheap to swap. It is reported separately because its benefit depends on the units being in that list.

6 Results on the unseen test photos

Every field in the 14 test photos was compared with the hand-checked transcription, after ignoring spaces and letter case. Exact match is the headline, because a value that is *almost* right (2.14 vs 2.4) is still wrong.

Configuration	Test name	Value	Unit	Ref. range	All	CER
A brief + oracle boxes	82.5	88.7	21.0	82.8	67.1	0.133
B tuned + oracle boxes	92.5	93.5	21.0	86.2	71.2	0.105
C brief pipeline (YOLO)	82.5	91.9	21.0	82.8	68.0	0.135
D tuned pipeline (YOLO)	87.5	93.5	21.0	86.2	70.3	0.107
E tuned + unit lexicon (YOLO)	87.5	93.5	88.7	86.2	89.2	0.036

Exact match in %, CER = mean character error rate. Oracle = hand-checked boxes instead of YOLO's, which isolates the OCR stage.

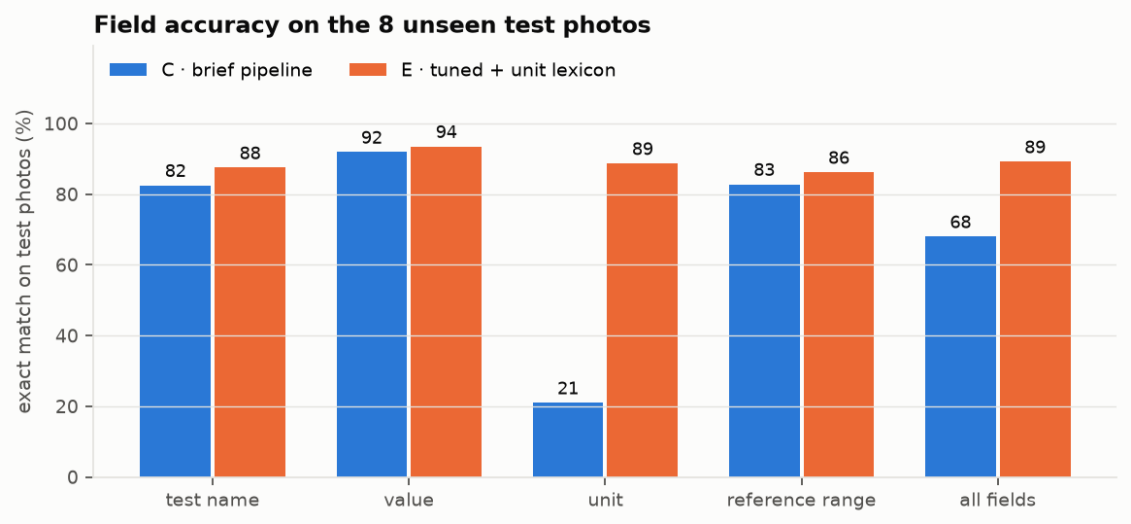


Figure 6. The brief's pipeline vs the shipped pipeline, field by field.

Final pipeline	liver	thyroid	vitamin
reference_range	90.9	95.2	0.0
test_name	90.9	33.3	100.0
unit	90.9	95.2	62.5
value	100.0	100.0	50.0
ALL	93.2	93.9	54.2

Exact match (%) by report type. The vitamin page is the hard case: 6-8 px print and a four-line reference range. Few thyroid test names are scored, because most thyroid photos crop them at the frame edge.

Is it safe to automate?

Of 62 test rows, **19.4%** passed straight through with no flag, and **91.7%** of those were fully correct. The review flag caught **18 of 19** rows that contained an error. YOLO produced no spurious rows. In production, the error-escape rate (unflagged rows with a mistake) is the number to drive towards zero, not overall accuracy.

7 What is still wrong

24 of 222 scored fields are wrong in the shipped pipeline. The most frequent errors:

Field	Truth	Read as	n
unit	pg/ml	pa/mi	2
reference_range	< 0.3	<0.	1
reference_range	0.3-5.5	0.3-S.5	1
reference_range	< 0.3	<O.%	1
reference_range	DEFICIENCY : <20 ng/ml | INSUFFICIENCY : 20-<30	DEFICIENCY 20 ng/mi | INSUFFICIENCY . 20-<30 ng/	1
reference_range	DEFICIENCY : <20 ng/ml | INSUFFICIENCY : 20-<30	DEFICIENCY | <20 ngiirs | INSUFFIQIENCY ; 20-<30	1
reference_range	< 0.3	<0O.%	1
reference_range	Normal : 211 - 911 pg/ml	Mormal ; 21t - 911 poym	1
reference_range	Normal : 211 - 911 pg/ml	Woemal : 231 - 91 poym	1
test_name	PROTEIN - TOTAL	PROTEIN - TOTAI	1

Three patterns account for the remaining errors. (1) **The vitamin page's 6-8 px print**: Tesseract misreads 4 of its 8 values (551 as 351 or 552, 21 as 2 or 22), 'pg/ml' as 'pa/mi', and never reads the four-line reference range exactly. **All four misread values were flagged for review.** (2) **Characters lost at the right edge of a tight box**: 'U/l' read as 'U/' or 'u/', '< 0.3' as '<0.'. (3) **Single-letter confusions** in test names: 'TOTAL' as 'TOTAI', '(T4)' as '(14)'. Pattern 2 suggests a cheap fix (a slightly wider right margin for unit and range crops). It was deliberately not tried here, because these errors were found on the test photos.

8 Limitations and next steps

- **Three documents.** The detector has learned three layouts. A new lab format needs about 5-10 annotated photos; the homography tool makes that fast.
- **Small test set.** 14 photos = 222 scored fields. One field is about 0.7 points, so differences under ~3 points are within noise.
- **OCR engine.** Tesseract is the limit on 6-8 px print. Next step: compare PaddleOCR and a fine-tuned TrOCR on value and unit crops.
- **Clinical sanity checks.** Flag values far outside their own reference range, and parse ranges into numeric low/high columns.
- **AWS not yet run on a live account.** The SageMaker scripts are tested offline (dry-run API requests, and the Docker image against SageMaker's /ping, /invocations and training-job file contracts), but have not been executed with real AWS credentials.

Reproduce

Step	Command / notebook
Audit + split	notebooks/01_setup_and_data_audit.ipynb
Labels	python tools/build_dataset.py --images datasets/raw

Step	Command / notebook
Train	python tools/train.py --weights yolov3u.pt --device 0 --batch 8 (Colab GPU)
Tune OCR	python tools/tune_preprocessing.py --images datasets/raw --workers 8
Compare	python tools/compare_detectors.py --weights model/yolov3u_best.pt model/yolo26n_best.pt
Run	python Custom_OCR.py --image <photo or folder>
Evaluate	notebooks/05_evaluation_and_optimisation.ipynb
Web app	uvicorn app.server:app --port 8000 or docker run -p 8080:8080 custom-ocr serve
AWS	notebooks/06_aws_sagemaker_s3.ipynb · python aws/sm_jobs.py --help

Privacy: the source photos show a patient's name, age, phone number and lab IDs. They are not published; every image in this report and the repository is redacted, and the OCR extracts only the four test fields.